Artificial Intelligence — Viva Voce Master Guide

Complete Technical Walkthrough, Theoretical Foundations & Professor Q&A Defense

Course: Artificial Intelligence | **Assignment: AI-GA1** | **Codebase: q1.py | q2.py | q3.py | q4.py**

Topics: BFS, DFS, Greedy, A*, Minimax, Alpha-Beta, Hill Climbing

AI Group Assignment 01: Complete Viva Voce Master Preparation Guide

Target Submission Folder: `iteration 1/AI-GA1/`
Files Covered: `q1.py`, `q2.py`, `q3.py`, `q4.py`
Reference Syllabus: `AI_GROUP_ASSIGNMENT_01_2026.pdf`

Quick Viva Strategy & Golden Rules

1. **Never hesitate on data structures:**
   - BFS = FIFO Queue (`collections.deque` or `list.pop(0)`)
   - DFS = LIFO Stack (`list.pop()`)
   - A* & Greedy = Priority Queue / Ordered Frontier
   - Minimax = Recursion Tree (Call Stack)
   - Hill Climbing = State Array (In-place evaluation)
2. **Always mention the Search Space & Branching Factor:**
   - Q1 Grid: Max branching factor $b \leq 4$, max depth $\leq R \times C$.
   - Q2 Graph: Branching factor = node degree, depth = path length.
   - Q3 Game Tree: Branching factor ≤ 4 per ply, tree depth D.
   - Q4 State Space: Total configurations = $S^N = 4^6 = 4096$, Neighborhood size = $N(S-1) = 6 \times 3 = 18$.
3. **Know the "Why":** Why did you choose this data structure? Why reverse order in DFS? Why $\alpha \geq \beta$ instead of $>$? Why strictly less cost in Hill Climbing?

Table of Contents

1. Question 1: Rescue Robot Grid Search (BFS vs DFS)
   - 1.1 Problem Overview & State Representation
   - 1.2 Line-by-Line Code Explanation (`q1.py`)
   - 1.3 Time & Space Complexity Breakdown
   - 1.4 Deep Viva Concepts (The Stack Reverse-Push Nuance, Graph vs Tree Search)
   - 1.5 Anticipated Viva Questions & Model Answers
2. Question 2: Campus Graph Route Finding (Greedy Best-First vs A*)
   - 2.1 Problem Overview & Graph Formulation
   - 2.2 Line-by-Line Code Explanation (`q2.py`)
   - 2.3 Time & Space Complexity Breakdown

- 2.4 Deep Viva Concepts (Admissibility, Consistency, Why Greedy Failed on Test Case 2)
- 2.5 Anticipated Viva Questions & Model Answers

3. Question 3: Robot Strategy Battle (Minimax & Alpha-Beta Pruning)
   - 3.1 Problem Overview & Game Formulation
   - 3.2 Line-by-Line Code Explanation (`q3.py`)
   - 3.3 Time & Space Complexity Breakdown
   - 3.4 Deep Viva Concepts (Pruning Mechanics, Move Ordering Impact, Heuristic Advantage)
   - 3.5 Anticipated Viva Questions & Model Answers
4. Question 4: University Exam Timetable Optimization (Hill Climbing Local Search)
   - 4.1 Problem Overview & CSP Formulation
   - 4.2 Line-by-Line Code Explanation (`q4.py`)
   - 4.3 Time & Space Complexity Breakdown
   - 4.4 Deep Viva Concepts (Steepest-Ascent, Plateaus, Ridges, Termination Reason)
   - 4.5 Answers to the 5 PDF Assignment Discussion Questions
   - 4.6 Anticipated Viva Questions & Model Answers
5. Grand Viva Rapid-Fire Defense Cheatsheet

Question 1: Rescue Robot Grid Search (BFS vs DFS)

File: `q1.py`

1.1 Problem Overview & State Representation

- **Objective:** Find a collision-free path for a rescue robot from starting cell `S` to goal cell `G` on an $R \times C$ grid with obstacles (`#`).
- **State Representation:** Coordinate tuple `(r, c)` where $0 \le r < R$ and $0 \le c < C$.
- **Actions:** 4 cardinal directions: UP $(r-1, c)$, RIGHT $(r, c+1)$, DOWN $(r+1, c)$, LEFT $(r, c-1)$.
- **Action Cost:** Uniform cost of 1 per move.
- **Successor Exploration Order:** Strictly enforced as: **UP → RIGHT → DOWN → LEFT**.

1.2 Line-by-Line Code Explanation (`q1.py`)

A. Input Parsing & Coordinates Extraction (Lines 4–25)

```
first_line = input().strip()
parts = first_line.split()
if len(parts) >= 2:
    R, C = int(parts[0]), int(parts[1])
elif len(parts) == 1 and len(parts[0]) == 2:
    R, C = int(parts[0][0]), int(parts[0][1])
```

- **What it does:** Reads grid dimensions R and C. Robustly handles both whitespace-separated input (e.g., `5 5`) and contiguous string inputs (e.g., `55`).
- **State initialization:** Iterates through every cell in the grid to identify the exact coordinates of start `S` (`start_r, start_c`) and goal `G` (`goal_r, goal_c`).

B. Direction Vectors (Lines 27–29)

```
dir_names = ["UP", "RIGHT", "DOWN", "LEFT"]
dr = [-1, 0, 1, 0]
dc = [0, 1, 0, -1]
```

- **What it does:** Defines the change in row (`dr`) and column (`dc`) corresponding to each action name.
- Index 0: UP $\rightarrow (-1, 0)$
- Index 1: RIGHT $\rightarrow (0, +1)$
- Index 2: DOWN $\rightarrow (+1, 0)$
- Index 3: LEFT $\rightarrow (0, -1)$

C. Breadth-First Search (BFS) Execution (Lines 31–56)

```
queue = [(start_r, start_c, [])]   # (row, col, path so far)
visited = set()
visited.add((start_r, start_c))
```

- **Frontier:** FIFO Queue implemented as a Python list with `queue.pop(0)`.
- **Visited Tracking:** A hash set (`set()`) for $O(1)$ amortized membership lookups.
- **Why mark visited at generation?** In BFS, marking a node visited as soon as it is enqueued prevents duplicate nodes from being added to the queue, saving massive memory and unnecessary expansion.

```
while len(queue) > 0:
    r, c, path = queue.pop(0)
    bfs_expanded += 1
    if r == goal_r and c == goal_c:
        bfs_found = True
        bfs_path = path
        break
    for i in range(4):
        nr = r + dr[i]
        nc = c + dc[i]
        if 0 <= nr < R and 0 <= nc < C and grid[nr][nc] != '#' and (nr, nc) not in visited:
            visited.add((nr, nc))
            queue.append((nr, nc, path + [dir_names[i]]))
```

- **Loop invariant:** Explores nodes in non-decreasing order of path length (level-by-level).
- **Expansion condition:** If current cell equals `(goal_r, goal_c)`, search terminates immediately.
- **Successor generation:** Explores in forward order $i \in [0, 1, 2, 3]$ (UP → RIGHT → DOWN → LEFT), pushing valid, unvisited, non-obstacle neighbors into the queue.

D. Depth-First Search (DFS) Execution (Lines 71–97)

```
stack = [(start_r, start_c, [])]
visited2 = set()
visited2.add((start_r, start_c))
```

- **Frontier:** LIFO Stack using `stack.pop()`.

```
# Add in REVERSE order so that UP is popped first from stack
for i in range(3, -1, -1):
    nr = r + dr[i]
    nc = c + dc[i]
    if 0 <= nr < R and 0 <= nc < C and grid[nr][nc] != '#' and (nr, nc) not in visited2:
        visited2.add((nr, nc))
        stack.append((nr, nc, path + [dir_names[i]]))
```

- **The Reverse Order Trick:** A stack is Last-In, First-Out (LIFO).
- If we want UP (index 0) to be evaluated first, it must be on **top** of the stack.
- Therefore, we push in reverse: index 3 (LEFT), index 2 (DOWN), index 1 (RIGHT), and finally index 0 (UP).
- When `stack.pop()` is called in the next iteration, UP is popped first!

1.3 Time & Space Complexity Breakdown

Algorithm	Time Complexity (Big-O)	Space Complexity (Big-O)	Completeness	Optimality
BFS	$O(R \times C)$	$O(R \times C)$	Complete (always finds path if exists)	**Optimal** (guaranteed shortest path for unit step cost)
DFS	$O(R \times C)$	$O(R \times C)$	Complete (on finite state space with visited set)	**Not Optimal** (returns first found path, can be long and winding)

Mathematical Derivation:

1. **Time Complexity:**
   - In the worst case, every cell in the $R \times C$ grid is visited once.
   - For each visited cell, 4 neighbors are inspected: $4 \times (R \times C)$ operations.
   - Hence, Time Complexity $= O(V+E) = O(R \cdot C + 4R \cdot C) = O(R \cdot C)$.
   - In tree-search terminology: $O(b^d)$, where $b \leq 4$ is the branching factor and d is the shallowest goal depth.
2. **Space Complexity:**
   - **BFS:** The queue stores the wavefront of the search. In the worst case (e.g., diagonal wave in open grid), queue holds $O(\min(R, C))$ or $O(R \cdot C)$ nodes. The `visited` set stores up to $R \times C$ entries. Total Space $= O(R \times C)$.
   - **DFS (Graph Search):** Because we maintain a `visited` set of size up to $R \times C$, space is $O(R \times C)$. In standard Tree-Search DFS without a visited set, space would be $O(b \cdot m)$ where m is maximum depth, but could get trapped in infinite loops.

1.4 Deep Viva Concepts

- **Why did BFS and DFS give the same path in Sample 1?**
  In Sample 1, the obstacles forced both BFS and DFS along the exact same corridor. Because the enforced movement order (UP → RIGHT → DOWN → LEFT) aligned with the only available open corridor, DFS happened to stumble upon the optimal path directly without deep divergence.
- **Why does BFS guarantee optimality?**
  Because all edge weights are equal ($c = 1$). BFS expands nodes strictly in non-decreasing order of path length ($g(n)$). The first time the goal node is reached, it is guaranteed to have the minimum possible number of moves.
- **Why is list `pop(0)` in Python $O(N)$?**
  Python lists are dynamic arrays. Removing the 0-th element shifts all subsequent elements to the left by one

position. In production or large grids, `collections.deque` with `popleft()` is preferred because it runs in $O(1)$ time.

1.5 Anticipated Viva Questions & Model Answers

Q1: Why did you push successors to the stack in reverse order (`range(3, -1, -1)`) in DFS?

> *"Because a stack is a Last-In, First-Out (LIFO) structure. The question requires successors to be explored in the order UP → RIGHT → DOWN → LEFT. If we pushed UP first, it would sit at the bottom of the stack and be popped last. By pushing LEFT first, then DOWN, then RIGHT, and finally UP, UP is placed at the top of the stack and is popped and expanded first."*

Q2: What is the difference between Graph Search and Tree Search in BFS/DFS?

> *"Tree search does not remember previously visited states; if cycles exist, it can enter infinite loops. Graph search maintains a `visited` (closed) set to ensure no state is expanded more than once. Our implementation uses Graph Search with a hash set."*

Q3: Can DFS ever be faster than BFS?

> *"Yes. If the goal happens to lie along the very first deep branch that DFS explores, DFS will reach it immediately with minimal node expansions and zero wavefront memory overhead, whereas BFS must exhaustively explore all intermediate depth levels."*

Question 2: Campus Graph Route Finding (Greedy Best-First vs A*)

File: `q2.py`

2.1 Problem Overview & Graph Formulation

- **Objective:** Find a route from start node A to goal node F on a weighted campus graph using two informed search strategies:
  1. **Greedy Best-First Search:** Evaluates nodes solely using the heuristic: $f(n) = h(n)$.
  2. **A* Search:** Balances path cost so far and estimated remaining cost: $f(n) = g(n) + h(n)$.
- **Graph Topology:** Bidirectional weighted edges:

$$A - B(4)$$

,
$$A - C(2)$$

,
$$B - D(5)$$

,
$$C - D(3)$$

,
$$C - E(6)$$

,
$$D - E(3)$$

,
$E-F(2)$

.
- **Heuristic Table to Goal F:**

$h(A)=7, h(B)=8, h(C)=5, h(D)=4, h(E)=2, h(F)=0$

.

2.2 Line-by-Line Code Explanation (`q2.py`)

A. Input & Adjacency List Construction (Lines 4–41)

```
adj = {}
# For each edge u, v with cost c:
adj[u].append((v, c))
adj[v].append((u, c))
```

- **Representation:** Adjacency List using a Python dictionary where each key maps to a list of `(neighbor, weight)` tuples.
- **Bidirectional handling:** Edges are added in both directions ($u \to v$ and $v \to u$).

B. Greedy Best-First Search Implementation (Lines 60–94)

```
frontier = [(start, [start], 0)]   # (current_node, path, cost_so_far)
visited = set()
while len(frontier) > 0:
    best = 0
    for i in range(1, len(frontier)):
        if h[frontier[i][0]] < h[frontier[best][0]]:
            best = i
    curr, path, cost = frontier.pop(best)
```

- **Selection Rule:** Iterates through `frontier` to find the node with minimum heuristic value $h(n)$.
- **Visited Check:** If `curr in visited: continue`. When popped, `visited.add(curr)`.
- **Termination:** When `curr == goal`, returns immediately.
- **Path Found on Sample 1:** `A -> C -> E -> F`, Cost = 10, Nodes Expanded = 4.

C. A* Search Implementation (Lines 110–152)

```
frontier2 = [(start, [start], 0)]   # (current_node, path, g_cost)
visited2 = set()
while len(frontier2) > 0:
    best = 0
    best_f = frontier2[0][2] + h[frontier2[0][0]]
    for i in range(1, len(frontier2)):
        f_i = frontier2[i][2] + h[frontier2[i][0]]
        if f_i < best_f:
            best = i
            best_f = f_i
        elif f_i == best_f:
            if len(frontier2[i][1]) > len(frontier2[best][1]):
                best = i
                best_f = f_i
    curr, path, g_cost = frontier2.pop(best)
```

- **Selection Rule:** Computes $f(n)=g(n)+h(n)$ for every node in the frontier and selects the minimum $f(n)$.
- **Tie-Breaking Rule:** If two nodes have equal $f(n)$, the code selects the one with the longer path length (`len(frontier2[i][1]) > len(frontier2[best][1])`), favoring deeper, more explored paths.
- **Path Found on Sample 1:** `A -> C -> D -> E -> F`, Cost = 10, Nodes Expanded = 5.

2.3 Time & Space Complexity Breakdown

Algorithm	Evaluation Function	Time Complexity (Worst-Case)	Space Complexity	Optimality Guarantee
Greedy Best-First	$f(n)=h(n)$	$O(bm)$ (or $O(\|V\|2)$ with list frontier)	$O(\|V\|)$	**No** (can be fooled by cheap heuristics into expensive paths)
A* Search	$f(n)=g(n)+h(n)$	$O(bd)$ (exponential in worst case)	$O(\|V\|)$ (stores all generated nodes in memory)	**Yes** (if $h(n)$ is admissible for tree search, and consistent for graph search)

- **Why $O(|V|2)$ in our code?**
  Our frontier uses an unsorted Python list with a linear scan $O(|V|)$ to find the minimum element. With N nodes, popping takes $O(|V|2)$. If implemented using a binary min-heap (`heapq`), extraction is $O(\log|V|)$ and total time is $O(|E|+|V|\log|V|)$.

2.4 Deep Viva Concepts

A. Admissibility vs Consistency (Monotonicity)

- **Admissible Heuristic:** Never overestimates the true cost to reach the goal:

$$h(n)\leq h^*(n), \forall n$$

(where $h^*(n)$ is the true optimal cost from n to goal).

- **Consistent (Monotonic) Heuristic:** For every node n and every successor n' generated by action a:

$$h(n) \leq c(n, a, n') + h(n')$$

(Satisfies the triangle inequality).
- **Proof that Campus Graph heuristic is Consistent:**
- $A \to C: h(A) = 7 \leq c(A,C) + h(C) = 2 + 5 = 7$ (Holds!)
- $C \to D: h(C) = 5 \leq c(C,D) + h(D) = 3 + 4 = 7$ (Holds!)
- $C \to E: h(C) = 5 \leq c(C,E) + h(E) = 6 + 2 = 8$ (Holds!)
- $D \to E: h(D) = 4 \leq c(D,E) + h(E) = 3 + 2 = 5$ (Holds!)
- $E \to F: h(E) = 2 \leq c(E,F) + h(F) = 2 + 0 = 2$ (Holds!)

Since it satisfies the triangle inequality across all edges, the heuristic is **consistent and therefore admissible**.

B. Why Did Greedy and A* Take Different Routes in Sample 1?

- At node C, the available neighbors are D and E.
- Greedy looks **only** at h: $h(E) = 2 < h(D) = 4$. Greedy immediately leaps to E, yielding path $A \to C \to E \to F$ (Cost = $2 + 6 + 2 = 10$).
- A* computes $f = g + h$:
  - For D: $g(D) = 2 + 3 = 5 \implies f(D) = 5 + 4 = 9$.
  - For E: $g(E) = 2 + 6 = 8 \implies f(E) = 8 + 2 = 10$.
  - Since $f(D) = 9 < f(E) = 10$, A* chooses D first!
- Both paths fortuitously ended up with equal cost 10, but Greedy expanded fewer nodes (4 vs 5).

C. Sample Test Case 2 (Demonstrating Greedy's Suboptimality)

- In the assignment's second test case (`p2_test_case_2.txt`):
- A deceptive branch offers a very low heuristic value $h = 1$, but the edge cost is massive (e.g., 100).
- **Greedy Best-First** gets trapped: it chases the tempting low h and selects the disastrous route.
- **A*** evaluates $g(n) + h(n)$: it detects that the high $g(n)$ makes $f(n)$ too large, and instead selects the true low-cost path.

2.5 Anticipated Viva Questions & Model Answers

Q1: What happens if an admissible heuristic is used in A* Graph Search, but it is NOT consistent?

> *"If a heuristic is admissible but inconsistent, A* might discover a shorter path to a previously closed node later in the search. To guarantee optimality in graph search with an inconsistent heuristic, the algorithm must allow re-opening of closed nodes when a strictly cheaper $g(n)$ is discovered. If the heuristic is consistent, the first time a node is expanded, its $g(n)$ is guaranteed to be optimal, and closed nodes never need to be reopened."*

Q2: Why does Greedy Best-First Search often expand fewer nodes than A*?

> *"Greedy Best-First Search is myopic: it aggressively rushes directly toward the goal by always picking the node that looks closest ($h(n)$). When the heuristic is accurate and there are no deceptive dead-ends, it behaves like a laser beam and reaches the goal with minimal expansions. However, it completely ignores the accumulated cost $g(n)$, sacrificing optimality."*

Q3: What is the effect of setting $h(n) = 0$ for all nodes in A*?

> *"When $h(n)=0$, $f(n)=g(n)+0=g(n)$. A* degenerates into Uniform Cost Search (Dijkstra's Algorithm). It is still guaranteed to find the optimal path, but it will expand nodes in concentric cost contours without any directional guidance toward the goal."*

Question 3: Robot Strategy Battle (Minimax & Alpha-Beta Pruning)

File: `q3.py`

3.1 Problem Overview & Game Formulation

- **Game Type:** Two-player, zero-sum, perfect information, turn-taking game on a grid.
- **Players:**
- MAX robot: `A` (strives to maximize the evaluation score).
- MIN robot: `B` (strives to minimize MAX's advantage).
- **Entities:** Obstacles (`#`), Empty cells (`.`), Energy cells (`E` worth $+10$ points).
- **Search Depth:** Depth-limited tree search up to depth D.

3.2 Line-by-Line Code Explanation (`q3.py`)

A. State Representation & Board Parsing (Lines 20–38)

```
a_pos = (r, c)
b_pos = (r, c)
energy_cells = set() # Coordinates of remaining energy cells
```

- **State Tuple:** `(a_pos, b_pos, a_score, b_score, energy, is_max_turn, d)`
- Current coordinates of both robots.
- Cumulative scores of both robots.
- A `set` of remaining energy cell coordinates.
- Turn indicator (`is_max_turn`) and remaining depth `d` .

B. Successor Function & Move Ordering (Lines 45–60)

```
def get_moves(pos, board):
    moves = []
    r, c = pos
    for i in range(4):
        nr, nc = r + dr[i], c + dc[i]
        if 0 <= nr < R and 0 <= nc < C and board[nr][nc] != '#':
            moves.append((dir_names[i], (nr, nc)))
    return moves
```

- Generates legal moves in the strict default order: UP → RIGHT → DOWN → LEFT.

```
def order_moves(moves, energy, use_heuristic):
    if not use_heuristic:
        return moves
    collecting = [m for m in moves if m[1] in energy]
    non_collecting = [m for m in moves if m[1] not in energy]
    return collecting + non_collecting
```

- **Heuristic Move Ordering:** Segregates moves into those that step onto an energy cell (`m[1] in energy`) versus ordinary moves. It executes energy-capturing moves first!

C. Evaluation Function (Lines 63–78)

```
def evaluate(a_pos, b_pos, a_score, b_score, energy, board):
    val = a_score - b_score
    if len(energy) > 0:
        best_a = 999
        best_b = 999
        for (er, ec) in energy:
            dist_a = abs(a_pos[0] - er) + abs(a_pos[1] - ec)
            dist_b = abs(b_pos[0] - er) + abs(b_pos[1] - ec)
            if dist_a < best_a:
                best_a = dist_a
            if dist_b < best_b:
                best_b = dist_b
        val += (best_b - best_a)
    return val
```

- **Formula:**

$$\text{Eval} = (\text{SMAX} - \text{SMIN}) + (\text{distMIN} - \text{distMAX})$$

- **Intuition:**
  1. Base component: Point difference between MAX and MIN.
  2. Positional Advantage: Manhattan distance to the nearest energy cell.
- If MAX is closer to energy than MIN ($\text{distMAX} < \text{distMIN}$), then $(\text{distMIN} - \text{distMAX}) > 0$, increasing MAX's score!

D. Minimax Algorithm (Lines 84–127)

- Standard recursive implementation:
- If `is_max_turn` : starts with `best_val = -infinity` , computes $\max$ over all child values.
- If `not is_max_turn` : starts with `best_val = +infinity` , computes $\min$ over all child values.
- Tracks `minimax_generated` (children created) and `minimax_expanded` (nodes whose subtree was entered).

E. Alpha-Beta Pruning Implementation (Lines 144–208)

```
# At MAX node:
if val > best_val:
    best_val = val
    best_move = move_name
if best_val > alpha:
    alpha = best_val
if alpha >= beta:
    pruned += 1
    break # Beta Cutoff!

# At MIN node:
if val < best_val:
    best_val = val
    best_move = move_name
if best_val < beta:
    beta = best_val
if alpha >= beta:
    pruned += 1
    break # Alpha Cutoff!
```

- α: Best value guaranteed for MAX along path so far (lower bound).
- β: Best value guaranteed for MIN along path so far (upper bound).
- **Cutoff condition:** As soon as $\alpha \geq \beta$, the current player knows that the opponent can force a better option elsewhere, so the remaining sibling branches are pruned!

3.3 Time & Space Complexity Breakdown

Algorithm	Move Ordering	Time Complexity (Big-O)	Effective Branching Factor	Space Complexity
Standard Minimax	Any	$O(b^d)$	b	$O(b \cdot d)$ (call stack)
Alpha-Beta (Worst Case)	Unfavorable (worst moves first)	$O(b^d)$	b (no pruning occurs)	$O(b \cdot d)$
Alpha-Beta (Best Case)	Optimal (best moves examined first)	$O(b^{d/2})$	b	$O(b \cdot d)$

- **Practical Consequence:** In the best case, Alpha-Beta can search **twice as deep** as Minimax in the same amount of computation time!

3.4 Deep Viva Concepts

- **What is the difference between Nodes Generated and Nodes Expanded?**
- **Node Generated:** A legal successor state is created and pushed to be considered (instantiated).
- **Node Expanded:** The algorithm actually enters that state, evaluates it, or generates its children.
- Nodes that are pruned inside the loop are never expanded!
- **Why $\alpha \geq \beta$ and not $\alpha > \beta$?**
  In a zero-sum deterministic game, if $\alpha = \beta$, MIN already has a move that limits MAX to at most β, and MAX already

has an alternative branch guaranteeing at least α. Since MAX cannot achieve strictly higher than β, further search along this sub-branch cannot yield a strictly better result. Pruning on $\geq$ saves unnecessary expansions without missing any better move.

3.5 Anticipated Viva Questions & Model Answers

Q1: Does Alpha-Beta Pruning change the final move or minimax value compared to vanilla Minimax?

> *"No! Alpha-Beta Pruning is mathematically guaranteed to return the exact same evaluation score and optimal move as full Minimax. It only prunes branches that are provably suboptimal and can have no influence on the final decision."*

Q2: How does Heuristic Move Ordering improve Alpha-Beta efficiency?

> *"Alpha-Beta relies on tightening the $[\alpha, \beta]$ window as early as possible. If we evaluate the best moves first (such as immediately capturing an energy cell), α jumps up rapidly at MAX nodes and β drops rapidly at MIN nodes. This causes $\alpha \geq \beta$ to trigger much earlier, pruning large subtrees before they are even visited."*

Q3: What is the Horizon Effect in depth-limited game playing?

> *"The horizon effect occurs when a serious threat or unavoidable loss is pushed just beyond the search depth limit D. The algorithm might make seemingly stalling moves believing it avoided the penalty, when in reality the catastrophe was simply hidden behind the horizon. Quiescence search is typically used to mitigate this by extending search depth for unstable/tactical states."*

Question 4: University Exam Timetable Optimization (Hill Climbing Local Search)

File: `q4.py`

4.1 Problem Overview & CSP Formulation

- **Objective:** Schedule $N=6$ courses into $S=4$ examination slots to minimize student exam conflicts and avoid overcrowding slots.
- **Variables:** Courses $C1, C2, C3, C4, C5, C6$.
- **Domains:** Slots $\{1, 2, 3, 4\}$.
- **Total State Space:** $S^N = 4^6 = 4096$ possible timetables.
- **State Representation:** Array of length N: `[slot_C1, slot_C2, slot_C3, slot_C4, slot_C5, slot_C6]`.
- Example: `[1, 1, 2, 3, 4, 2]` means $C1 \rightarrow$ Slot 1, $C2 \rightarrow$ Slot 1, ...

4.2 Line-by-Line Code Explanation (`q4.py`)

A. Objective Cost Function (Lines 10–30)

```
def calculate_cost(state, conflicts, S):
    # 1. Conflict penalty: 10 per conflicting pair in same slot
    num_conflicts = 0
    for a, b in conflicts:
        if state[a - 1] == state[b - 1]:
            num_conflicts += 1
    conflict_penalty = num_conflicts * 10

    # 2. Distribution penalty: 2 per excess course beyond 2 in any slot
    slot_count = [0] * (S + 1)
    for slot in state:
        slot_count[slot] += 1

    total_excess = 0
    for s in range(1, S + 1):
        if slot_count[s] > 2:
            total_excess += slot_count[s] - 2
    distribution_penalty = total_excess * 2

    total_cost = conflict_penalty + distribution_penalty
    return total_cost, conflict_penalty, distribution_penalty
```

- **Cost Formula:**

$$\text{Total Cost} = (10 \times \text{Number of Conflicts}) + (2 \times \sum_{s=1}^{S} \max(0, \text{Count}(s) - 2))$$

- **Soft Constraints:**
- Conflicts: If two conflicting courses share the same slot, penalty $= 10$.
- Distribution Capacity: Up to 2 courses per slot are free. Each additional course incurs a penalty of 2.

B. Neighborhood Generation & Steepest-Ascent Logic (Lines 62–77)

```
# Generate neighbors by changing slot of exactly one course
for course in range(N):
    for slot in range(1, S + 1):
        if slot != current[course]:
            neighbor = current[:]
            neighbor[course] = slot
            cost, conf_pen, d_pen = calculate_cost(neighbor, conflicts, S)
            neighbors_this_round += 1

            # Tie-breaking: select first such neighbor generated
            if cost < best_neighbor_cost:
                best_neighbor = neighbor[:]
                best_neighbor_cost = cost
                best_conflict = conf_pen
                best_dist = d_pen
```

- **Neighborhood Size:** For every course ($N = 6$), we can change its slot to any of the other $(S - 1 = 3)$ slots:

$$\text{Number of Neighbors} = N \times (S - 1) = 6 \times 3 = 18 \text{ neighbors}$$

- **Steepest-Ascent Selection:** Evaluates all 18 neighbors in each iteration.

- **Tie-Breaking Rule:** The condition `if cost < best_neighbor_cost:` uses strict inequality. If multiple neighbors share the same minimum cost, the **first one generated** is preserved.

C. Stopping Condition & Evaluation Counting (Lines 78–95)

```
total_neighbors_evaluated += neighbors_this_round

# Stop if no neighbor is strictly better
if best_neighbor is None:
    break
```

- **Strict Improvement Requirement:** The algorithm transitions only if `cost < current_cost`. If no neighbor has a lower cost, `best_neighbor` remains `None` and the loop breaks.
- **Evaluation Counting Requirement:** As specified in Section 14 of the assignment PDF, the final iteration (where all 18 neighbors are evaluated and none are better) **must be included** in the total count of evaluated states.
- 2 improving iterations $+1$ stopping iteration $=3\times18=54$ evaluations!

4.3 Time & Space Complexity Breakdown

Metric	Theoretical Complexity	Assignment Value ($N=6, S=4, K=8$)
Neighborhood Size per Iteration	$O(N\cdot S)$	$6\times(4-1)=18$ states
Cost Function Evaluation	$O(K+S+N)$	$8+4+6=18$ operations
Time Complexity per Iteration	$O(N\cdot S\cdot(K+S))$	$\approx18\times18=324$ operations
Total Run Time (for I iterations)	$O(I\cdot N\cdot S\cdot(K+S))$	For 2 iterations $+1$ check: <0.001 seconds
Space Complexity	$O(N+K+S)$	Negligible (<1 KB in RAM)

4.4 The 3 Experimental Runs & Comparison Table

Running `q4.py` on the three required initial timetables:

Exp #	Initial State	Initial Cost	Final State	Final Cost	Iterations	Neighbors Evaluated	Execution Time	Termination Reason
1	`[1, 1, 2, 3, 4, 2]`	20	`[3, 1, 1, 3, 4, 2]`	**0**	2	54	≈0.00021 s	Local Optimum
2	`[2, 3, 1, 1, 4, 4]`	20	`[2, 3, 3, 1, 2, 4]`	**0**	2	54	≈0.00021 s	Local Optimum
3	`[4, 2, 3, 1, 2, 3]`	20	`[4, 2, 2, 1, 4, 3]`	**0**	2	54	≈0.00012 s	Local Optimum

4.5 Answers to the 5 PDF Assignment Discussion Questions

1. **Which initial timetable produced the best final solution?**
   All three initial timetables produced an optimal solution with a final cost of 0 (zero conflicts and zero excess distribution penalty). Each achieved the ideal objective.
2. **Did all initial states reach the same final timetable?**
   No! Each experiment converged to a different permutation of course assignments (e.g., Exp 1: `[3, 1, 1, 3, 4, 2]`*, Exp 2:* `[2, 3, 3, 1, 2, 4]`*, Exp 3:* `[4, 2, 2, 1, 4, 3]`*). Because the timetable problem has symmetric optimal configurations across the 4 slots, hill climbing finds the nearest local/global basin of attraction.*
3. **Did Hill Climbing get stuck in different local optima?**
   In these three specific runs, Hill Climbing did not get stuck in a suboptimal local optimum; all three reached global optima with cost = 0. However, in larger problem instances, Hill Climbing frequently gets trapped in suboptimal local minima or flat plateaus.
4. **How did the initial state affect the number of iterations?**
   All three test cases required exactly 2 improvement iterations because each initial configuration was two single-variable moves away from a zero-conflict configuration.
5. **How did the initial state affect the final cost?**
   In this small state space, the initial state did not degrade the final cost—all converged to cost 0. In general CSPs, poorly chosen initial states can lead to suboptimal local optima with higher final costs.

4.6 Anticipated Viva Questions & Model Answers

Q1: What are the main failure modes of Hill Climbing?

> *"1. Local Minima/Optima: Points where all neighbors have higher cost, but a better global solution exists elsewhere.*
> *2. Plateaus / Flat Regions: Areas where all neighbors have identical cost, making gradient guidance impossible.*
> *3. Ridges: Narrow sequences of local peaks where single-variable moves cannot make progress without moving diagonally across two variables simultaneously."*

Q2: How can we overcome local optima in Local Search?

> *"1. Random-Restart Hill Climbing: Runs hill climbing repeatedly from random initial states and keeps the best result.*
> *2. Simulated Annealing: Allows occasional bad (uphill) moves with a probability* $P = e^{-\Delta E / T}$ *that decreases over time as temperature* T *cools.*
> *3. Tabu Search: Keeps a memory list of recently visited states to prevent cycling back into visited basins.*
> *4. Genetic Algorithms: Maintains a population of solutions and combines them using crossover and mutation."*

Q3: Why is our algorithm called 'Steepest-Ascent' (or Steepest-Descent)?

> *"Because it exhaustively evaluates ALL* $N(S-1) = 18$ *immediate neighbors in the neighborhood and selects the neighbor that provides the steepest improvement in cost. In contrast, 'First-Choice Hill Climbing' generates neighbors randomly and accepts the first neighbor that improves the cost."*

Grand Viva Rapid-Fire Defense Cheatsheet

If the Examiner Asks...	Deliver this Instant 1-Sentence Answer:
"Why do we use heuristics in search?"	To reduce the effective branching factor and prune the search space by estimating the distance to the goal.
"Why not use BFS for everything?"	BFS suffers from exponential memory consumption ($O(bd)$) because it must store all active wavefront nodes in RAM.
"Is DFS complete?"	In graph search with a visited set on a finite graph, yes; in tree search on graphs with cycles, no, because it can loop indefinitely.
"What makes A optimal?"*	An admissible heuristic (never overestimating) ensures A* never overlooks a cheaper path before declaring the goal found.
"Why does Alpha-Beta prune?"	If a player already has a guaranteed outcome that is better than what the current sub-branch can provide, exploring further is provably irrelevant.
"What is the time complexity of Alpha-Beta in the best case?"	$O(bd/2)$, which effectively doubles the search depth achievable in a given time limit compared to minimax.
"Why does Hill Climbing not require a search tree?"	It is a Local Search algorithm; it does not care about the path taken, only the final state configuration. Memory is $O(1)$ relative to search depth.
"Why are conflicts given penalty 10 and excess courses penalty 2?"	Course conflicts are hard constraints (two exams at the same time prevent students from taking both); room capacity is a softer constraint.
"How many neighbors does Question 4 evaluate per round?"	Exactly $N \times (S-1) = 6 \times (4-1) = 18$ neighbors.
"What was the termination reason in Question 4?"	Local Optimum (no single-course reassignment could strictly reduce the cost below the current state).

Created for AI Group Assignment 01 Viva Voce Defense. Good luck with your viva!